CHAPTER 1

INTRODUCTION

1.1. Background of the Project

In recent years, the increasing number of road accidents has become a major public safety concern worldwide. A critical factor influencing survival in accident cases is the time taken to provide medical assistance. Delays in accident reporting, ambulance dispatch, and reaching hospitals often result in severe injuries or loss of life. Traditional emergency response systems rely heavily on manual communication, static routing, and limited coordination between emergency services, ambulances, and hospitals, which reduces their effectiveness during time-critical situations.

With rapid advancements in web technologies, GPS tracking systems, mapping services, and data-driven decision-making, it is now possible to build intelligent emergency response systems. Technologies such as mapping APIs, traffic monitoring, Artificial Intelligence, and machine learning enable dynamic route optimization, injury analysis, and faster decision-making. Rescue Now – an intelligent emergency response and ambulance management system – emerges from this need to improve coordination, reduce response times, and ensure timely medical care.

The project integrates accident reporting, ambulance dispatch, hospital route optimization, AI-based wound detection, and doctor–patient matchmaking into a single web-based platform. By identifying the nearest available ambulance, selecting the appropriate hospital, assessing injury severity, and notifying suitable doctors in advance, the system enhances the efficiency of emergency medical services. Overall, Rescue Now aims to save lives by transforming traditional emergency response into a smart, automated, and reliable system aligned with the digital transformation of healthcare services.

1.2. Problem Context

In many road accident cases, the lack of an efficient emergency response mechanism leads to unnecessary delays in medical treatment. Accident reporting is often delayed

due to confusion or the absence of immediate communication channels. Even after an accident is reported, ambulances may not be dispatched optimally, and drivers may follow longer or congested routes due to limited awareness of current traffic conditions.

Existing emergency systems generally rely on static routing methods that fail to adapt to real-time conditions such as traffic congestion, roadblocks, or unexpected incidents. Additionally, there is often poor coordination between ambulances, hospitals, and doctors, resulting in delays in patient admission and treatment preparation. The absence of injury severity assessment further limits medical preparedness at hospitals.

Furthermore, emergency service operators face difficulties in managing multiple incidents simultaneously due to the lack of centralized dashboards and real-time tracking mechanisms. These challenges highlight the need for an integrated, intelligent, and real-time solution capable of automating emergency workflows, detecting injury severity, and supporting rapid decision-making during high-pressure situations.

1.3. Problem Statement

To design a smart emergency response system that enables rapid accident reporting, efficient ambulance allocation, optimized hospital routing, AI-based wound detection, and doctor–patient matchmaking to reduce delays in providing critical medical assistance.

1.4. Motivation for the Project

The motivation for this project arises from the urgent need to reduce emergency response times and improve survival rates in road accident scenarios. Many lives are lost each year due to delays caused by inefficient ambulance dispatch, traffic congestion, lack of injury assessment, and poor coordination between emergency responders, hospitals, and doctors. Traditional systems fail to adapt dynamically to changing conditions, making them unreliable during critical emergencies.

Another motivating factor is the increasing availability of GPS-based tracking, real-time traffic data, and Artificial Intelligence techniques that can be effectively used to optimize ambulance routing, injury assessment, and medical resource allocation. By

analyzing traffic patterns and wound severity, emergency services can make faster and more accurate decisions.

The project is also motivated by the need to improve communication and transparency among emergency stakeholders. Real-time updates, live ambulance tracking, wound severity alerts, and centralized monitoring dashboards can greatly enhance coordination and operational efficiency. Rescue Now is designed with the goal of creating a reliable, scalable, and life-saving system that leverages technology to support emergency medical services and save lives.

1.5. Objectives of the Project

1. To provide real-time accident reporting and tracking:

The system aims to enable quick and accurate reporting of road accidents with precise location details, ensuring that emergency services are alerted without delay.

2. To monitor ambulance locations using GPS technology:

The project focuses on continuous real-time tracking of ambulances to support efficient dispatch and monitoring.

3. To identify nearby hospitals based on distance and traffic conditions:

The system analyzes location and traffic data to select suitable hospitals for faster patient admission.

4. To minimize emergency response time using optimized routes:

Route optimization algorithms are used to calculate the fastest and safest routes dynamically.

5. To classify wound severity using AI-based techniques:

The system analyzes accident-related inputs to assess injury severity and prioritize medical response

6. To match patients with appropriate doctors:

Based on injury severity, the system identifies and alerts suitable medical specialists in advance.

1.6 Scope of the Project

In-Scope:

Accident Reporting System:

The system allows users or authorities to report road accidents through a web-based interface.

GPS-Based Ambulance Tracking:

Ambulance locations are tracked in real time to ensure efficient dispatch and monitoring.

Automatic Ambulance Dispatch:

The nearest available ambulance is automatically assigned to the accident location.

Hospital Identification and Route Optimization:

Nearby hospitals are identified and optimal routes are calculated using mapping APIs and traffic data.

AI-Based Wound Severity Detection:

The system analyzes injury information to classify wound severity for medical prioritization.

Doctor–Patient Matchmaking:

Appropriate doctors are notified in advance based on injury type and severity.

Admin Dashboard:

Emergency coordinators can monitor accidents, ambulances, hospitals, and doctor assignments in real time.

Out-of-Scope:

Autonomous Ambulance Operation:

The system does not support self-driving or autonomous ambulances.

Wearable Health Device Integration:

The project does not integrate with smartwatches or medical wearables.

Live Video Streaming:

Real-time video transmission from accident sites is not included.

Advanced Medical Diagnosis:

The system does not perform detailed medical diagnosis or treatment planning.

Telemedicine or Video Consultation:

Live doctor–patient consultations are outside the scope.

CHAPTER 2

LITERATURE SURVEY

Y.Chen, *Et.al* presented a smart health care application that integrates IoT and GPS to optimize emergency medical services. The system identifies the nearest hospital using GPS, calculates distances with the Haversine formula, and enables real-time communication between ambulances and hospitals via the Blynk app. This approach enhances ambulance response efficiency by reducing delays and improving coordination. However, the system requires enhancements such as traffic-aware routing, offline support and stronger data security to address its limitations and improve overall effectiveness **[1]**.

A.Al-Zahrani proposed a smart ambulance system that leverages IoT and AI technologies to optimize ambulance dispatch, routing and resource allocation. It utilizes real- time traffic data, road network information and historical data to predict traffic conditions and identify optimal routes. Additionally, it prioritizes emergency calls based on patient severity and condition, ensuring critical cases are addressed first **[2]**.

Azwati Azmin, *Et.al* provided an overview of existing smart ambulance systems, their technologies, and potential benefits. It emphasizes integrating IoT, AI, and big data analytics to improve the efficiency and effectiveness of emergency medical services. It also highlights challenges such as data privacy, security, and interoperability issues **[3]**.

Devigayathri P, *Et.al* presented a mobile-based ambulance management system that utilizes GPS and GIS for real-time tracking and shortest route identification using Dijkstra's algorithm. It includes features such as a panic button for emergencies, cloud storage for secure data management and accident detection using sensors and GSM for immediate alerts. While the system enhances emergency response and ensures efficient resource management, it could be further improved by incorporating traffic condition analysis, reducing dependence on internet connectivity, and addressing data security challenges **[4]**.

CHAPTER 3

SYSTEM ANALYSIS AND METHODOLOGY

3.1. System Analysis

3.1.1. Introduction

System analysis is a structured process used to study and evaluate an existing system to understand its functioning, limitations, and areas that require improvement. In the context of emergency medical services, system analysis helps identify inefficiencies in accident reporting, ambulance dispatch, injury assessment, and coordination with hospitals and doctors. By analysing how current emergency systems operate and how users interact with them, the actual needs and expectations of stakeholders are clearly understood.

The purpose of system analysis is to define what the proposed system must achieve to deliver faster, smarter, and medically effective emergency response. It helps in identifying functional and non-functional requirements, understanding emergency workflows, and determining how modern technologies such as GPS, Artificial Intelligence, and real-time traffic data can improve performance. This analysis ensures that the proposed system addresses real-world emergency challenges and provides a reliable, scalable, and life-saving solution.

3.1.2. Existing System

In the existing emergency response system, accident reporting is mostly manual and depends on phone calls or verbal communication, which often leads to delays and inaccurate location information. Ambulance dispatch is usually handled without real-time tracking, resulting in inefficient allocation of available ambulances. Drivers often follow fixed routes without considering current traffic conditions, increasing response time.

Existing systems do not provide any mechanism for early injury assessment. Hospitals and doctors are informed only after the patient arrives, leaving little time for treatment preparation. There is also poor coordination between emergency services, ambulances,

hospitals, and doctors. Emergency operators lack centralized dashboards to monitor multiple incidents simultaneously, making incident management difficult.

Overall, the existing system is slow, manual, error-prone, and lacks automation. It does not support real-time tracking, wound severity detection, or doctor–patient coordination, which significantly affects patient survival and quality of emergency care.

3.1.3. Proposed System

The proposed system, RescueNow – Intelligent Emergency Response System, is designed to overcome the limitations of existing manual and fragmented emergency response mechanisms by automating accident reporting, ambulance dispatch, injury analysis, and doctor coordination. Instead of relying on delayed phone calls and manual decision-making, the system enables real-time accident reporting, automatic ambulance allocation, optimized routing, and intelligent medical support. This significantly reduces response time, human error, and dependency on manual coordination.

The system integrates advanced technologies such as GPS tracking, real-time mapping APIs, Artificial Intelligence, and machine learning. When an accident is reported, the system captures the exact location and dispatches the nearest available ambulance automatically. Real-time traffic data is used to calculate the fastest route to the most suitable hospital. The system performs AI-based wound detection using uploaded symptoms or injury details. Based on the detected condition, the system determines severity levels such as mild, moderate, or severe. For mild cases, basic prescription recommendations are provided, and patients are given the option to book a doctor appointment. For moderate and severe cases, doctor–patient matchmaking is performed automatically, doctors are notified in advance, and appointments are booked. The system also supports smart token booking, enabling efficient hospital queue management. Compared to traditional systems, RescueNow is faster, automated, user - friendly, scalable, and medically intelligent, ensuring improved emergency care and saving lives.

3.1.4. Feasibility Study

The feasibility study evaluates whether the proposed RescueNow – Intelligent Emergency Response System can be realistically developed and successfully deployed in real-world emergency healthcare environments. The project is technically feasible as it relies on existing technologies such as GPS tracking, mapping APIs, AI-based wound detection, and web application frameworks, all of which are reliable and widely available. The system is operationally practical, as it provides simple interfaces for users, ambulance drivers, doctors, and administrators, making it suitable for emergency services and healthcare institutions.

3.1.4.1. Technical Feasibility

The project is technically feasible because all the required technologies such as GPS-based location tracking, real-time traffic APIs, Artificial Intelligence for wound detection, and doctor matchmaking algorithms are well supported and easily available. These technologies can be implemented using common programming languages and libraries, making system development and integration straightforward. The hardware requirements are minimal, and no specialized equipment is required, ensuring smooth implementation without major technical limitations.

3.1.4.2. Economic Feasibility

The project is economically feasible because it can be developed using cost -effective and mostly open-source technologies. The system does not require expensive hardware or proprietary software, and cloud services can be used only when required, keeping operational costs low. By automating ambulance dispatch, injury assessment, doctor allocation, appointment booking, and smart token generation, the system reduces manual effort and long-term operational expenses for emergency services and hospitals, making it a cost-effective solution.

3.1.4.3. Operational Feasibility

The proposed system is operationally feasible because it is designed to be easy to use and efficient, even for users with minimal technical knowledge. Emergency responders, ambulance drivers, doctors, and administrators interact with the system through intuitive dashboards and interfaces. By automating accident reporting, ambulance

tracking, wound detection, prescription viewing, appointment booking, and smart token management, the system significantly improves existing manual operations and enhances emergency response efficiency.

3.1.4.4. Time Feasibility

The project is time feasible because its development can be completed within the given semester or project duration. The required technologies—such as GPS APIs, AI models, and web frameworks—are readily available, allowing faster implementation without building components from scratch. The system can be developed in modular phases, ensuring steady progress and easy integration. Therefore, the project can be successfully completed within the allotted time frame.

3.2. Methodology

3.2.1. Introduction

Methodology refers to the structured plan and systematic approach followed in developing the proposed RescueNow system after completing the system analysis phase. It outlines how the system will be designed, implemented, and executed to achieve the identified objectives. A well-defined methodology ensures that the development process is organized, efficient, and follows a logical sequence of activities. It also ensures that the final system is reliable, scalable, and capable of meeting emergency healthcare requirements.

3.2.2. Project Approach

The project follows a systematic and structured approach beginning with requirement gathering, where emergency response challenges, user needs, and system limitations are clearly identified. Based on these requirements, the system design phase defines the architecture, workflow, modules, and interactions between system components. After finalizing the design, the implementation phase involves developing core functionalities such as accident reporting, GPS-based ambulance tracking, route optimization, AI-based wound detection, prescription generation, doctor–patient matchmaking, appointment booking, and smart token management. Once implementation is complete, the system undergoes testing to verify accuracy, performance, and reliability. Finally, evaluation is carried out using scenario-based

testing and performance metrics to confirm that the system meets its objectives and improves emergency response outcomes.

3.2.3. System Workflow

The system follows a structured workflow that begins with accident reporting and ends with patient treatment coordination. The complete process flow is described below:

1. User or authority opens the RescueNow application.
2. Accident details are reported.
3. The system captures the accident location using GPS.
4. The nearest available ambulance is identified.
5. Ambulance is dispatched automatically.
6. Traffic data is analyzed for route optimization.
7. Wounds and symptom data are processed using AI.
8. Wounds and symptom severity are identified.
9. Prescription recommendations are generated for mild cases.
10. Doctor–patient matchmaking is performed for moderate and severe cases.
11. Doctor appointment is booked and smart token is generated.
12. Admin monitors all activities through a centralized dashboard.

3.2.4. Tools and Technologies

The development of the proposed RescueNow – Intelligent Emergency Response System involves the following tools and technologies:

1. Programming Languages

- **Python** – Used for backend logic, AI processing, wound detection, doctor matchmaking, and system integration.
- **HTML, CSS, JavaScript** – Used for designing and developing the frontend user interface.

2. Backend Framework

- **Flask (Python)** – A lightweight web framework used to build server-side logic, handle user requests, manage processing tasks, and communicate with AI modules.

3. GPS and Mapping Tools

- **Google Maps API / OpenStreetMap** – Used for real-time location tracking and route optimization.

4. AI and Machine Learning Tools

- **Python AI/ML Libraries** – Used for wound severity detection, disease identification, and prescription recommendation.

5. Database

- **MongoDB** – Used to store accident data, ambulance status, hospital information, doctor profiles, appointments, prescriptions, and smart token details.

CHAPTER 4

REQUIREMENT SPECIFICATION

4.1. Introduction

Requirement Specification is a detailed document that clearly defines what the RescueNow system should do and how it is expected to perform. It outlines all functional and non-functional requirements, system features, constraints, and user expectations in an organized manner. This document acts as a blueprint for developers, designers, and stakeholders, ensuring a clear and common understanding of the system's objectives and behavior. Proper requirement specification helps avoid ambiguity, reduces development errors, and provides a reference throughout the project lifecycle. By clearly describing system requirements, it ensures that the final product meets emergency healthcare needs and supports efficient patient–ambulance–doctor coordination.

4.2. Functional Requirements

1. Accident Reporting

- The system shall allow users to report road accidents with real-time location details.
- The system shall validate the accident information before processing.

2. Ambulance Dispatch

- The system shall automatically identify and dispatch the nearest available ambulance using GPS data.
- The system shall notify users once an ambulance is successfully assigned.

3. Route Optimization

- The system shall calculate the fastest route between the accident location and the nearest hospital.
- The system shall dynamically update routes based on real-time traffic conditions.

4. Hospital Coordination

- The system shall identify suitable nearby hospitals based on distance and availability.
- The system shall notify hospitals in advance about incoming emergency patients.

5. Wound and Disease Detection

- The system shall analyze uploaded symptoms or wound details using AI techniques.
- The system shall classify severity levels such as mild, moderate, or severe.

6. Doctor–Patient Matchmaking

- The system shall recommend appropriate doctors based on detected symptoms or diseases.
- The system shall allow patients to book appointments and receive prescriptions.

7. Real-Time Updates

- The system shall provide real-time updates on ambulance location and emergency status.

8. User Authentication and Role Management

- The system shall support secure login for patients, doctors, and administrators.
- The system shall restrict access based on user roles.

9. Incident Logging and Reporting

- The system shall log all emergency incidents and medical interactions.
- The system shall allow admins to review historical data for analysis.

4.3. Non-Functional Requirements

1. Usability

- The system should be user-friendly and easy to operate for patients, doctors, and admins.
- Interfaces should provide clear instructions with minimal complexity.

2. Security

- All user data, medical records, and location details must be securely stored.
- Role-based access control should prevent unauthorized access.

3. Availability

- The system should be available 24/7 with minimal downtime.
- Emergency services must function even during peak usage.

4. Performance

- The system should process accident reports and ambulance dispatch requests quickly.
- Real-time tracking and notifications should not experience noticeable delays..

5. Reliability

- The system should function consistently without failure during emergencies.
- Backup and recovery mechanisms should ensure data integrity.

6. Scalability

- The system should support multiple simultaneous accident reports.
- It should scale to support future expansion across cities or regions.

7. Integration

- The system should integrate smoothly with GPS services, maps, and notification APIs.

8. Compliance

- The system should comply with healthcare and data protection standards.

4.4. Hardware Requirements

1. Processor

Minimum Intel Core i5

A multi-core processor is required to support backend processing, AI-based wound detection, and real-time data handling.

2. RAM

8 GB RAM

Adequate memory is required to manage concurrent requests, database operations, and AI computations efficiently.

3. Storage

500 GB HDD

Sufficient storage is needed to maintain logs, medical records, appointment data, and system backups.

4. Network

Stable Internet Connection

Reliable connectivity is required for GPS tracking, real-time updates, and API integration.

4.5. Software Requirements

1. Operating System

- Windows 11

  Used for development and deployment, supporting all required development tools and libraries.

2. Programming Language and Libraries

- **Python (Flask/Django)** – Backend logic, AI processing, and API handling.
- **HTML, CSS, JavaScript, React.**js – Frontend development.

3. Database

- **MongoDB** – Used for storing user data, hospital details, appointments, and incident logs.

4. APIs and Tools

- **GPS and Mapping APIs** – For location tracking and route optimization.
- **Notification APIs (SMS/Email)** – For real-time alerts.
- **Testing Tools** – Postman for API testing.

5. Frontend Tools

- **HTML, CSS, JavaScript** – For building the user interface and chatbot interactions.

CHAPTER 5

DESIGN

5.1. Introduction

System Design is the process of planning and defining the architecture, components, interfaces, and data flow of a system before actual development begins. It provides a clear blueprint of how the Rescue Now system will work internally, showing the interaction between different modules through diagrams and models such as Data Flow Diagrams (DFDs), UML diagrams, and architectural representations. System design is essential because it converts functional requirements into a structured framework, ensuring that developers clearly understand how emergency workflows, medical intelligence, and coordination mechanisms should be implemented. By establishing this visual and logical roadmap, system design reduces complexity, minimizes errors, and ensures that the final implementation is efficient, scalable, and aligned with real-world emergency healthcare requirements.

5.2. High-Level Design

5.2.1. Architectural Diagram

An Architectural Diagram is a high-level visual model that illustrates the overall structure of a software system, showing its major components, subsystems, and the interactions between them. It provides a blueprint that outlines how different parts of the system work together, how data flows between modules, and how the system is organized internally. This diagram helps developers, designers, and stakeholders understand the system's architecture before implementation, ensuring clarity in design decisions, scalability, maintainability, and proper integration of all components.

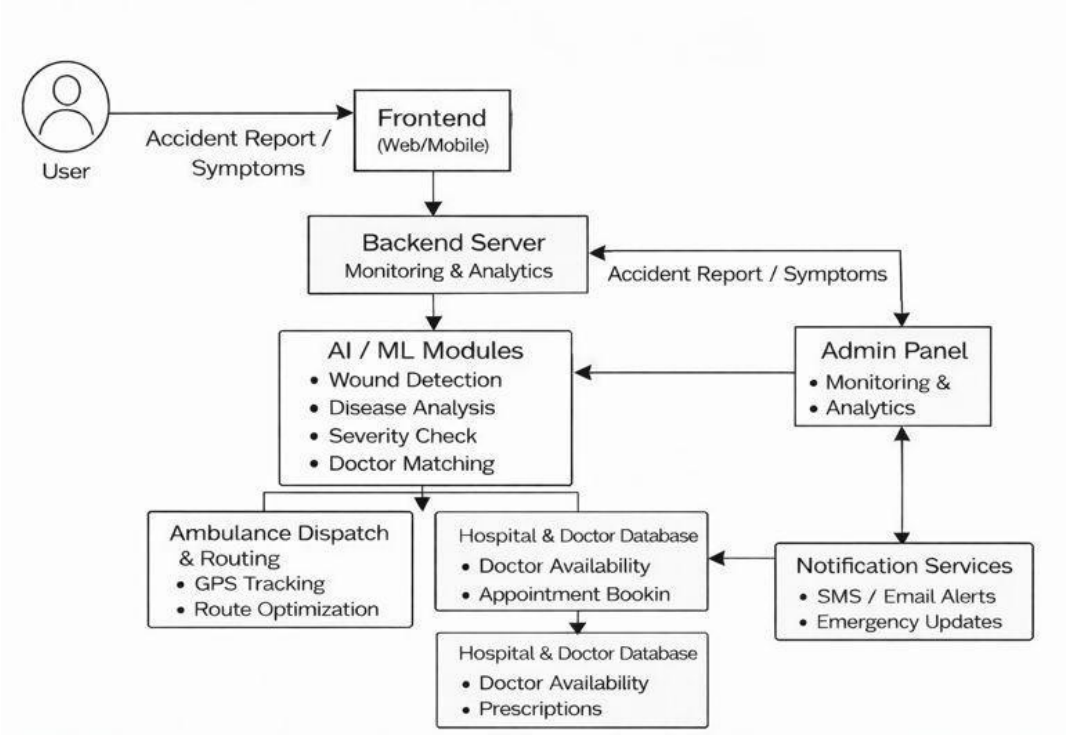


Figure 5.2.1: Architectural Diagram

The architectural diagram illustrates the high-level structure and workflow of the RescueNow Smart Healthcare Emergency Response System. It shows how the user interacts with the system through the frontend interface, where accident reports and symptoms are submitted using a web or mobile application. The frontend communicates with a centralized backend server, which serves as the core processing and control unit of the system while also handling monitoring and analytics functions. The backend forwards the received accident and symptom data to the AI/ML modules, which include wound detection, disease analysis, severity checking, and doctor matching. These intelligent components work together to analyze the patient's condition and determine appropriate medical actions. Based on the analysis, the system triggers ambulance dispatch and routing using GPS tracking and route optimization techniques. Simultaneously, the backend accesses the hospital and doctor database to retrieve doctor availability, manage appointment booking, and handle prescriptions. Notification services are used to generate emergency alerts and updates, which are reflected in the admin panel for monitoring and analytics purposes. Overall, the architectural diagram provides a clear overview of how the frontend, backend, AI

modules, emergency services, databases, and administrative components interact in a coordinated manner to ensure timely emergency response and effective healthcare management.

5.2.2. Context Diagram

A Context Diagram is a high-level visual model that represents the entire system as a single unified process and shows its interaction with external systems and supporting subsystems. It provides a simplified top-level view of how data flows into and out of the system without exposing internal processing details. The main purpose of a context diagram is to clearly define the system boundaries, identify connected systems, and illustrate the major inputs and outputs. This helps stakeholders understand the overall behavior of the system, how it communicates with related systems, and what role each external component plays before moving into more detailed design representations.

Symbol	Description
System Block	Represents any system component, including the main system and all functional subsystems.
Data Flow Line	Shows interaction, communication, or flow of information between the main system and its subsystems.
<<System>> **Stereotype Label**	Indicates that the block represents a system or subsystem within the architecture.

Table 5.2.1: Symbols Representation for Context Diagram

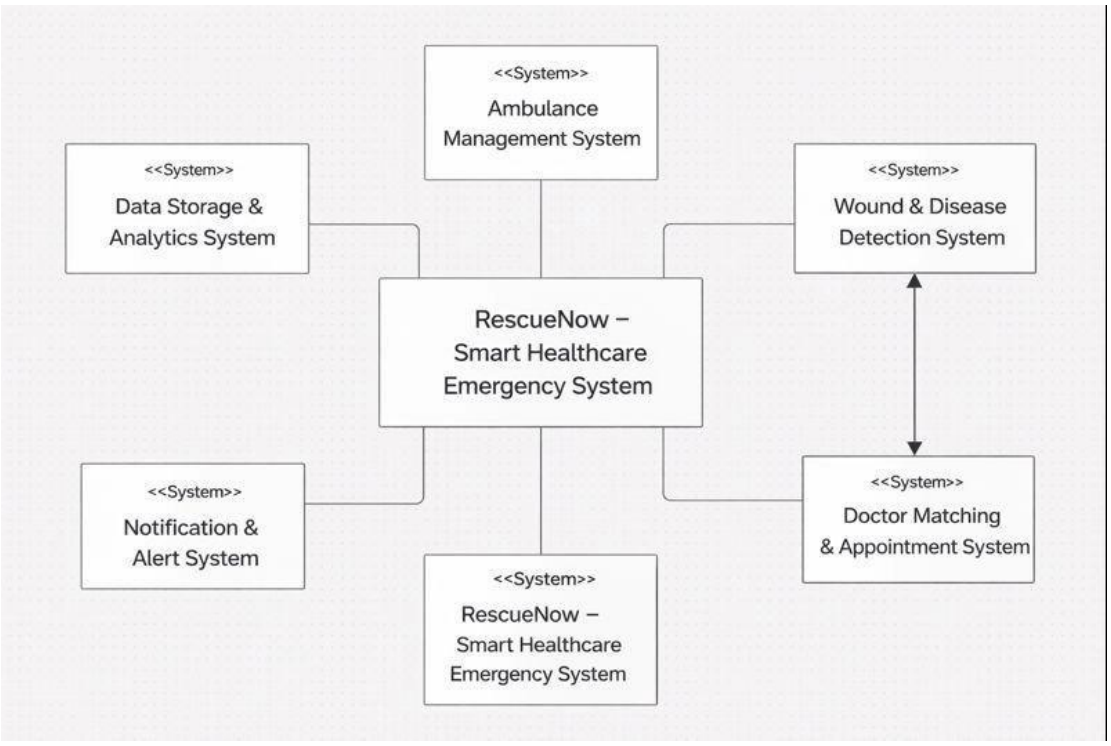


Figure 5.2.2: Context Diagram

The purpose of this diagram is to provide a clear understanding of how the RescueNow – Smart Healthcare Emergency System operates and how its major functional subsystems interact with the central emergency platform. It highlights the key supporting systems—such as ambulance management, wound and disease detection, doctor matching and appointment handling, notification and alert services, and data storage and analytics—and shows how each subsystem contributes to delivering a fast, intelligent, and coordinated emergency healthcare response.

Main Elements

1. **RescueNow – Smart Healthcare Emergency System:** The central system that manages and coordinates all emergency response activities, including accident reporting, ambulance dispatch, medical analysis, and communication.

2. Ambulance Management System: Responsible for locating available ambulances, tracking them using GPS, and managing dispatch and routing during emergencies.

3. Wound Detection System: Uses AI-based techniques to analyze patient symptoms or wound inputs, identify possible medical conditions, and assess severity levels.

4. **Doctor Matching & Appointment System:** Matches patients with appropriate doctors based on symptoms and severity, supports appointment booking, and enables smart token management.

5. Notification & Alert System: Sends real-time alerts and updates to patients, ambulance drivers, doctors, and hospitals through SMS.

6. Data Storage & Analytics System: Stores patient records, emergency logs, medical history, and operational data, and supports analytics for monitoring system performance and improving decision-making.

5.2.3. Activity Diagram

An Activity Diagram is a behavioral diagram that represents the workflow or sequence of actions in a system, showing how processes start, flow, and end. It illustrates the step-by-step activities, decisions, and parallel processes involved in completing a task.

Symbol	Description
Start Symbol	Starting point of the workflow
Activity Symbol	Represents an action performed in the system
Control Flow	Flow of control from one step to the next.
Stop Symbol	Stopping point of the workflow

Table 5.2.2: Symbols Representation for Activity Diagram

Figure 5.2.3: Activity Diagram

The purpose of this Activity Diagram is to illustrate the complete workflow of the RescueNow Smart Healthcare Emergency Response System, showing every major step involved in handling an emergency from accident reporting to treatment initiation. It visually represents the flow of actions starting from the user or patient reporting an accident or symptoms, logging accident details and location, dispatching an ambulance, tracking ambulance movement, and simultaneously analyzing patient symptoms using AI. The diagram highlights both sequential and parallel processes, helping to understand system behavior, task dependencies, and coordination between emergency response and medical preparation. This representation ensures clarity during system design, development, testing, and documentation.

Main Elements

1. Start Node: Begins the emergency workflow.

2. User Actions: User reports accident and updates symptoms**.**

3. Emergency Response: System locates and dispatches the nearest ambulance and tracks its movement.

4. AI Processing: System analyzes injury severity and matches a suitable doctor.

5. Dashboard Update: Emergency status is displayed for monitoring.

6. Hospital Processing: Doctor allocation and treatment initiation occur at the hospital**.**

7. End Node: Marks the completion of the emergency response process.

5.2.4. Use Case Diagram

A Use Case is a functional description of how a user or external actor interacts with a system to achieve a specific goal. It describes the sequence of actions, system responses, and interactions that occur during a particular emergency operation. Use cases help define system requirements from the user's perspective, ensuring that the system supports all necessary functionalities required for effective emergency response and coordination.

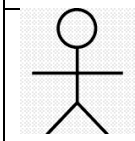
Symbol	Description
 **Actor**	Represents the external user interacting with the system.
System Use Case	Each oval shows an **action** the system provides.

Table 5.2.3: Symbols Representation for Use Case Diagram

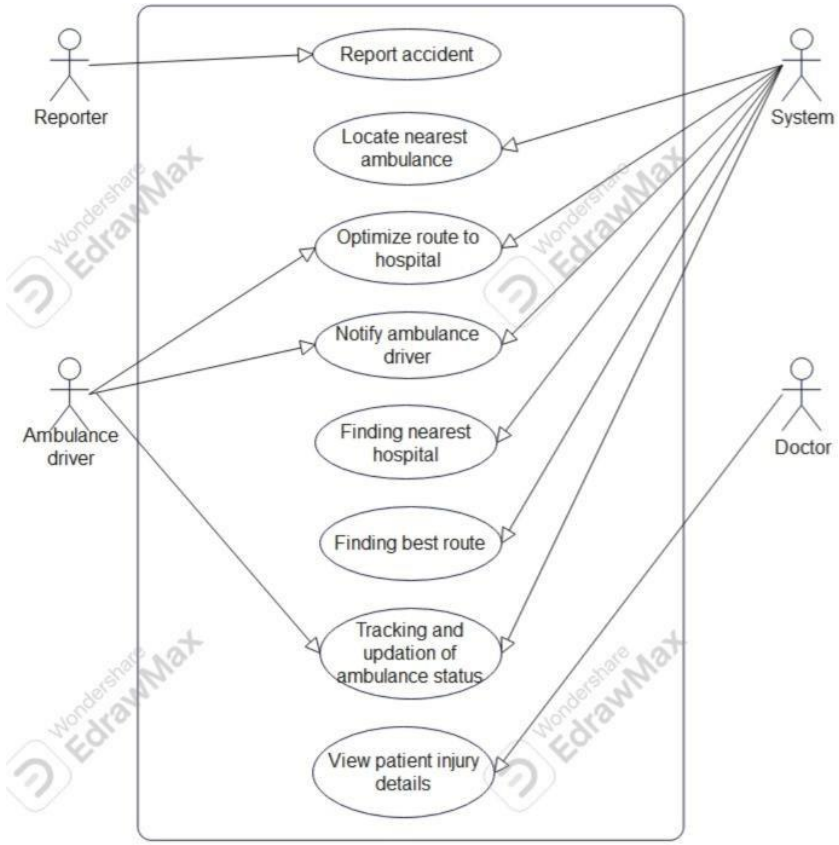


Figure 5.2.4: Use Case Diagram

The purpose of this Use Case Diagram is to visually represent the interactions between the different actors and the RescueNow Emergency Response System. It highlights how various users and system components participate in emergency management activities such as accident reporting, ambulance coordination, route optimization, hospital communication, and real-time status tracking. The diagram clearly shows the responsibilities of each actor and the services provided by the system, helping to understand functional requirements and user roles before system implementation. It acts as a clear blueprint of system behavior from an emergency response and coordination perspective.

Main Elements

1. **Reporter:** Reports the accident and initiates the emergency response process

2. **Ambulance Driver:** The ambulance driver receives notifications from the system, follows the optimized routes provided to reach the hospital efficiently, and continuously updates and tracks the ambulance status in real time during the emergency response.

3. **Doctor:** Views patient injury details to prepare for treatment.

4. **RescueNow System:** The system locates the nearest ambulance, identifies the nearest hospital, optimizes the best route to the hospital, notifies the ambulance driver, continuously tracks and updates the ambulance status, and manages the overall coordination of emergency response activities.

5.3 User Interface Design

User Interface (UI) design refers to the visual layout and arrangement of screens, buttons, icons, and interactive elements that users interact with in the RescueNow system. Its purpose is to present emergency-related information in a clear, organized, and visually understandable manner, making the system easy to operate during critical situations. A well-designed UI helps users quickly access essential features, understand system responses, and perform actions such as reporting emergencies or tracking ambulances without confusion. Overall, the UI design enhances usability and ensures a smooth, intuitive, and reliable experience for all users.

The user interface design of the RescueNow system outlines the overall flow of interaction between different pages to ensure efficient emergency handling and coordination. The process begins on the Home Page, where the user can report an emergency, enter patient details, and initiate ambulance dispatch. From this page, the user is directed to the Doctor / Hospital Page, which allows hospitals or doctors to assign medical professionals, set appointments, and manage prescriptions based on the patient's condition. The system then navigates to the Notification / Appointment Page, where users receive alerts, confirm appointments, and track ambulance movement in real time. Finally, the workflow concludes when the patient visits the hospital at the scheduled time

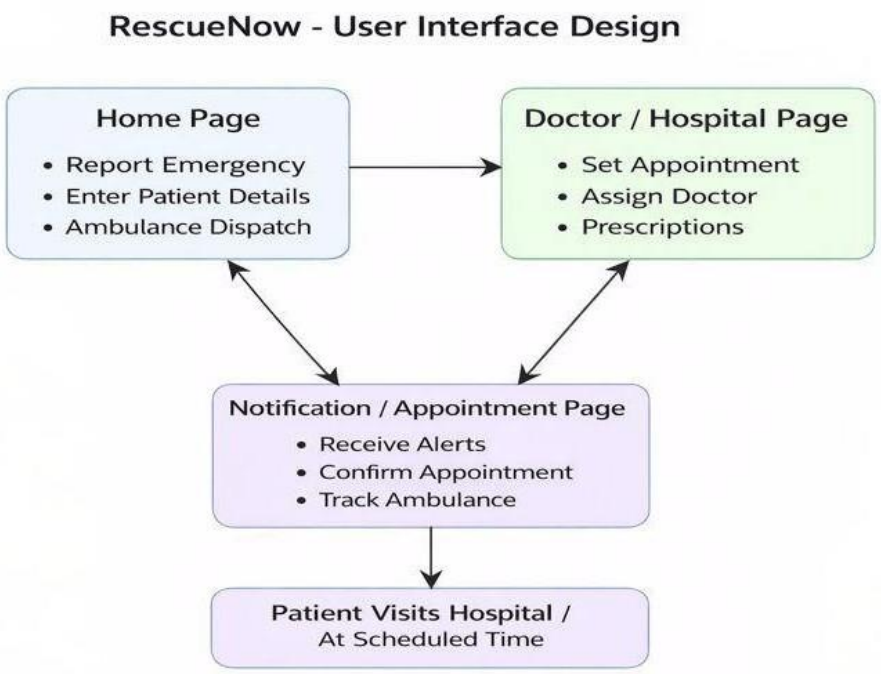


Figure 5.3.1: User Interface Design

The design ensures that all primary actions—emergency reporting, ambulance dispatch, doctor assignment, notifications, and tracking—are clearly distributed across intuitive screens. This structured layout enables users to move through the system effortlessly, even in high-pressure situations. The arrows in the diagram represent the logical flow of navigation, making the interface simple, guided, and user-friendly.

CHAPTER 6

IMPLEMENTATION

6.1 Introduction

Implementation is the phase in which the system design is converted into a working application. It involves developing the backend logic, integrating databases, connecting APIs, and ensuring smooth interaction between different modules. This phase transforms theoretical concepts into a real-time, usable system. During implementation, each module is tested individually and then integrated to ensure correct functionality, reliability, and performance. Proper implementation ensures that the system can be deployed and used effectively in real-world healthcare and emergency scenarios.

6.2. Modules and their Functionality

1. User Authentication Module

- Handles user registration and login for patients, doctors, and administrators.
- Uses secure password hashing for authentication.
- Maintains user sessions and role-based access control.

2. Emergency Reporting Module

- Allows users to report emergencies and medical conditions through the application.
- Captures user location details for emergency processing.
- Initiates ambulance booking and medical workflows.

3. Ambulance Booking and Dispatch Module

- Identifies the nearest hospital based on user GPS location.
- Calculates distance using geolocation techniques.
- Generates real-time navigation links using Google Maps.
- Sends ambulance booking confirmation via SMS notifications.

4. AI-Based Wound and Disease Detection Module

- Accepts patient symptoms and medical inputs.
- Uses AI techniques to analyze symptoms and identify possible diseases.
- Classifies medical conditions into mild, moderate, or severe categories.
- Supports early medical decision-making.

5. Prescription Recommendation Module

- Suggests basic medicines for mild conditions.
- Displays prescription details to patients.
- Allows patients to proceed with doctor consultation if required.
- Supports early medical decision-making.

6. Doctor–Patient Matchmaking Module

- Matches patients with suitable doctors based on symptoms and specialization.
- Ranks doctors using similarity analysis, experience, and workload.
- Displays recommended doctors to patients.
- Supports early medical decision-making.

7. Appointment Booking and Smart Token Module

- Enables patients to book doctor appointments online.
- Updates doctor availability and appointment count.
- Generates smart tokens for queue management.
- Reduces hospital waiting time.

8. Doctor Management Module

- Allows doctors to view appointments and patient details.
- Supports leave management and emergency reassignment.
- Enables doctors to add digital prescriptions.

9. Admin Management Module

- Allows administrators to register hospitals and doctors.

- Manages doctor availability and hospital data.
- Monitors system activity through an admin dashboard.

10. Notification Module

- Sends SMS alerts for ambulance booking using Twilio.
- Ensures real-time communication among system users.

11. Database Management Module

- Stores user data, medical history, appointments, prescriptions, and hospital details.
- Uses MongoDB for efficient and secure data management.
- Supports real-time data updates.

CHAPTER 7

TESTING

7.1. Introduction

Testing is an essential phase in software development that involves executing the system to verify whether it performs as expected and delivers the required functionality without errors. It ensures that every module, feature, and process in the system works correctly and according to the specifications defined during the requirement and design phases. Testing helps identify bugs, performance issues, and deviations from expected behavior before deployment. In RescueNow, testing ensures reliable emergency reporting, accurate ambulance dispatch, correct wound detection, proper doctor matching, and smooth appointment booking. Effective testing improves system quality and ensures safe and dependable real-world usage.

7.2. Testing Strategies

7.2.1. Unit Testing

Unit Testing is the process of testing individual functions or small units of code independently. In RescueNow, unit testing validates functions such as user authentication, GPS location capture, distance calculation, symptom analysis, and prescription generation. This testing ensures that each unit performs its intended task correctly and helps detect errors at an early stage.

7.2.2. Module Testing

Module Testing involves testing individual modules after integrating their internal units. In this system, modules such as ambulance booking, wound detection, doctor recommendation, appointment booking, and notification services are tested separately. This ensures that each module works correctly before integration with other modules.

7.2.3 Integration Testing

Integration Testing checks the interaction between different modules once they are combined. It verifies data flow and communication between components such as emergency reporting and ambulance dispatch, wound detection and doctor matchmaking, and appointment booking with notifications. This testing ensures smooth coordination across the system.

7.2.4 System Testing

System Testing evaluates the complete RescueNow system as a whole. It verifies whether all modules work together correctly and meet functional and non-functional requirements such as performance, reliability, usability, and security. This testing ensures that the system is stable and ready for real-world emergency scenarios.

7.3. Test Cases

7.3.1. Introduction

Test cases are a structured set of conditions used to verify whether specific system functions work as expected. Each test case includes defined inputs, test actions, and expected outputs. Test cases help validate system accuracy, reliability, and consiste ncy. In RescueNow, test cases ensure correct operation of emergency services, medical analysis, doctor matching, and appointment workflows.

7.3.2. Unit Test Cases

Test case ID	Input	Test Case Description	Expected Output	Status
UT_01	Invalid login credentials	Attempt login with wrong username/password	Error message displayed	Pass
UT_02	Invalid GPS coordinates	Submit incorrect location data	Location validation error shown	Pass
UT_03	Mild symptoms input	Submit symptoms with mild severity	Prescription suggestions displayed	Pass
UT_04	Empty symptom fields	Submit symptom form without data	Validation message shown	Pass

Table 7.3.1: Unit Test Cases

7.3.3. Module Test Cases

Test Case ID	Input	Test Case Description	Expected Output	Status
MT_01	Emergency report details	Verify that the system extracts text from selected PDF pages.	Extracted text displayed	Pass
MT_02	User GPS location	Check if the nearest ambulance is identified	Nearest ambulance dispatched.	Pass
MT_03	Patient symptoms	Validate AI-based wound and disease detection module	Severity and condition identified	Pass

Table 7.3.2: Module Test Cases

7.3.4. Integration Test Cases

Test case ID	Input	Test Case Description	Expected Output	Status
IT_01	Accident report → Ambulance module	Submit valid accident and location details	Ambulance dispatched successfully	Pass
IT_02	Symptoms form → Wound detection module	Submit patient symptoms	Disease and severity identified	Pass
IT_03	Appointment form → Notification service	Book doctor appointment	Confirmation message sent	Pass

Table 7.3.3: Integration Test Cases

7.3.5. System Test Cases

Test Case ID	Input	Test Case Description	Expected Output	Status
ST_01	Valid accident details	Report accident	Ambulance dispatched successfully	Pass
ST_02	Valid symptoms input	Perform wound and disease detection	Severity identified and displayed	Pass
ST_03	Valid appointment data	Book doctor appointment	Appointment confirmed	Pass
ST_04	Complete valid workflow	End-to-end emergency response flow	Successful execution	Pass

Table 7.3.4: System Test Cases

CHAPTER 8

RESULTS

System Output Screenshots:

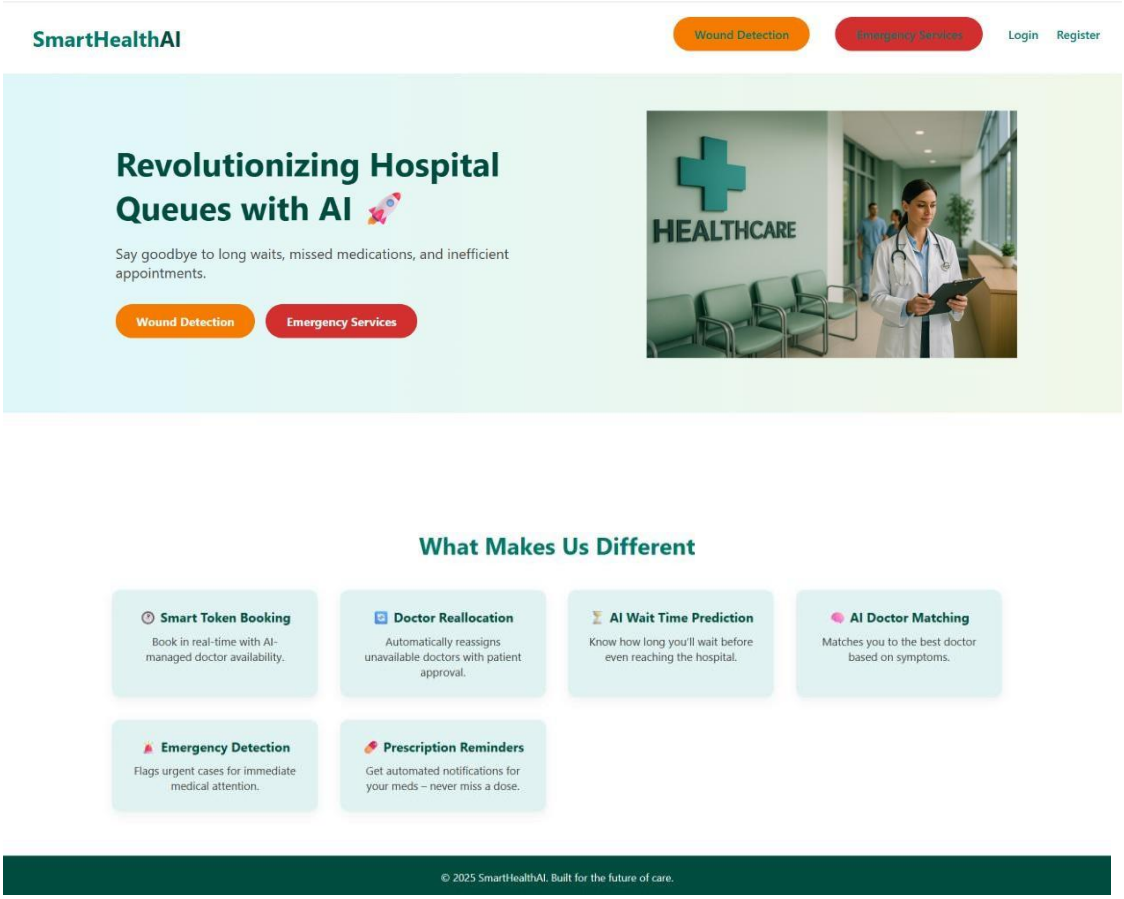


Figure 8.1: Smart-Health AI Homepage Interface

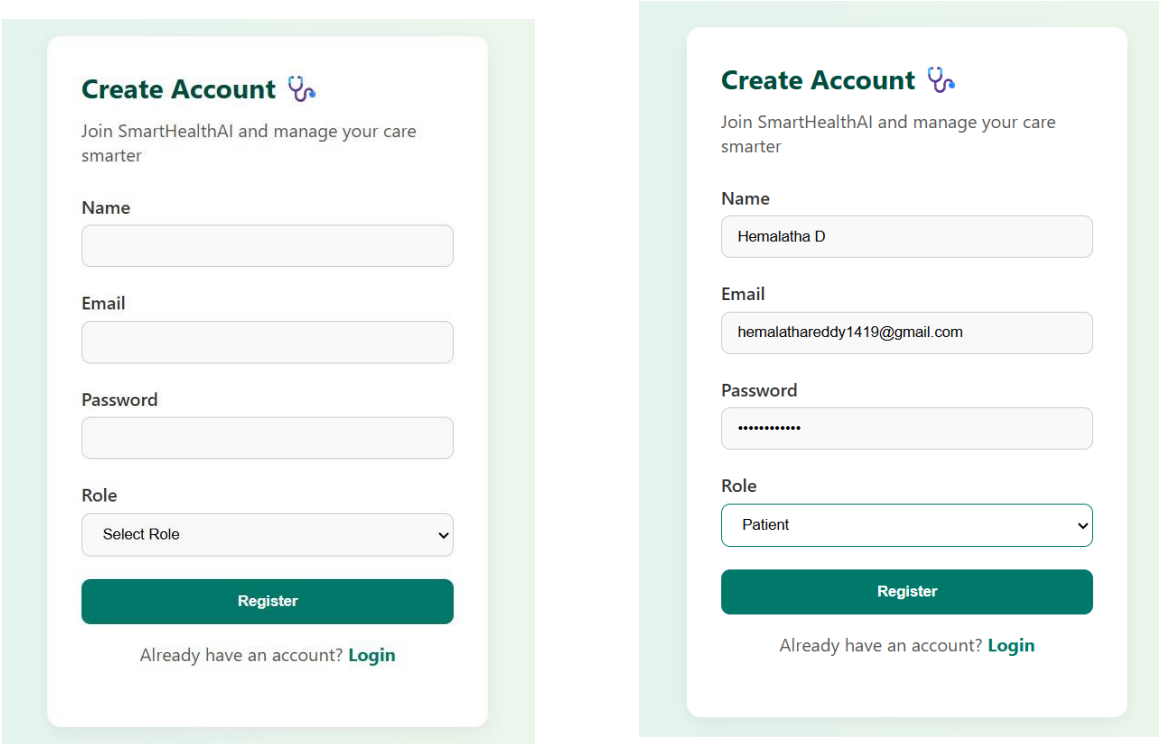


Figure 8.2: Register Interface

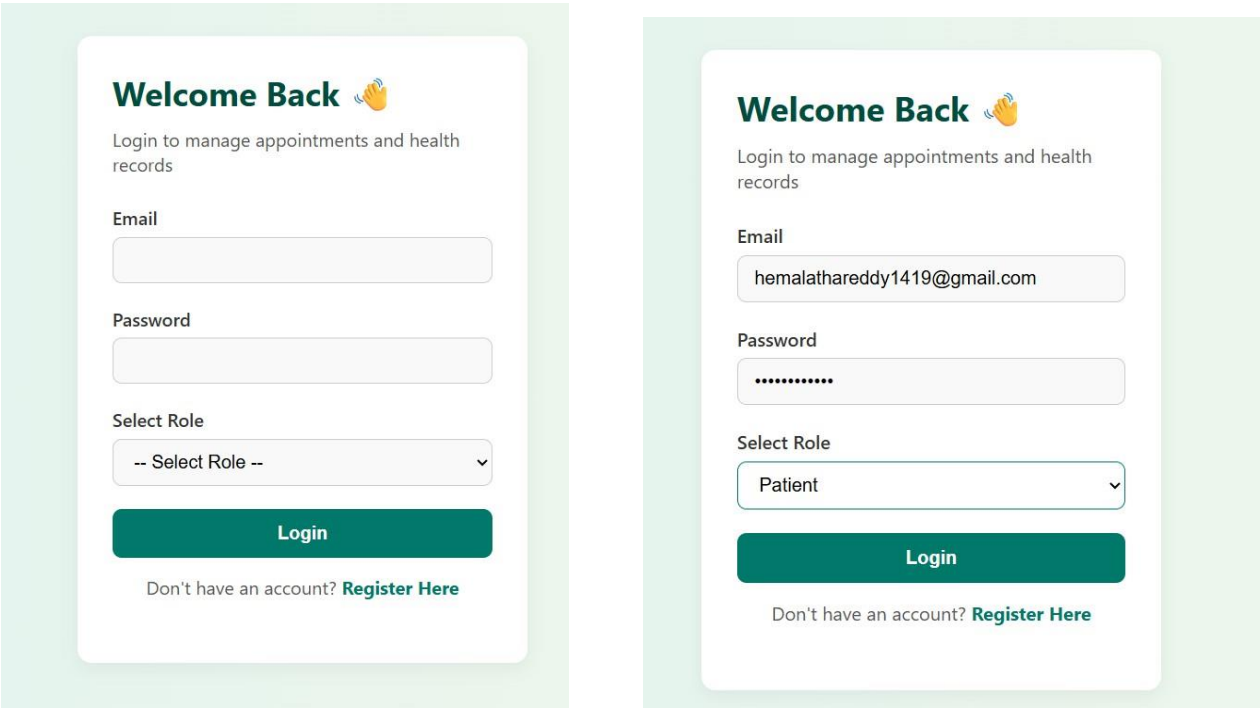


Figure 8.3: Login Interface

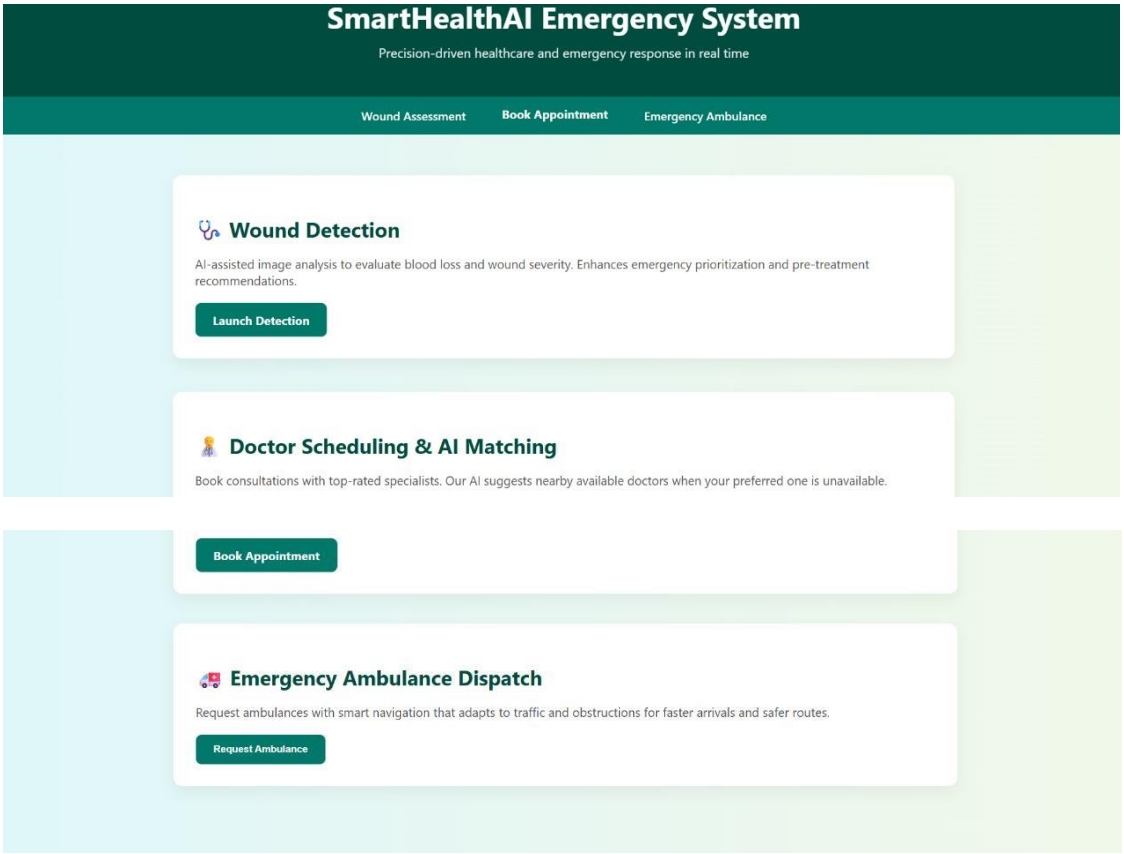


Figure 8.4: User Interface

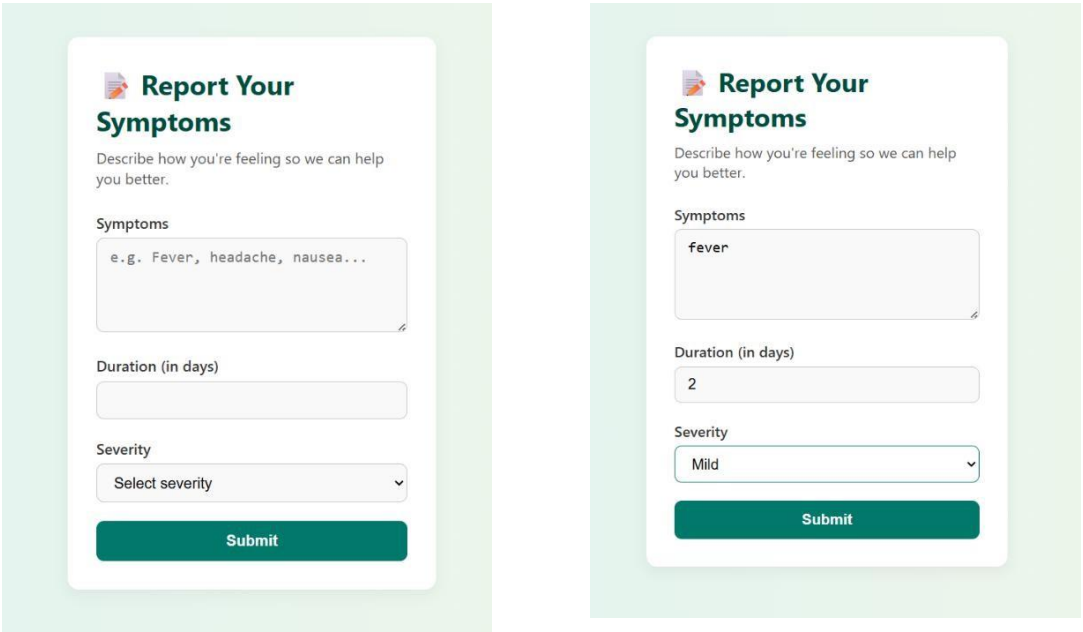


Figure 8.5: Report Symptoms Interface

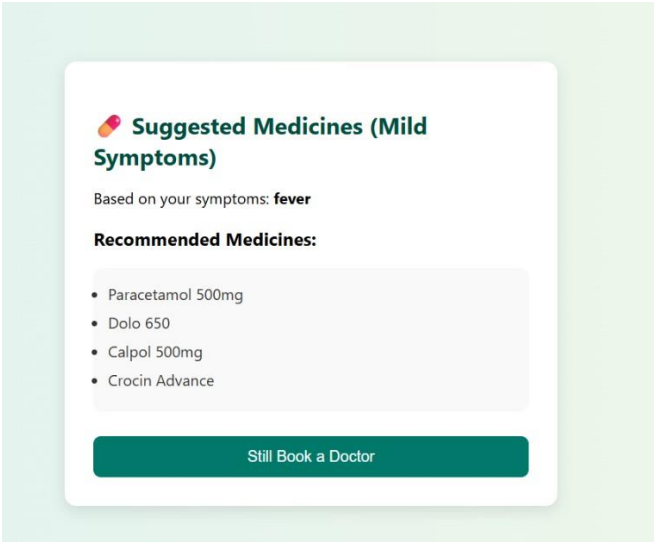


Figure 8.6: Medicines Suggested

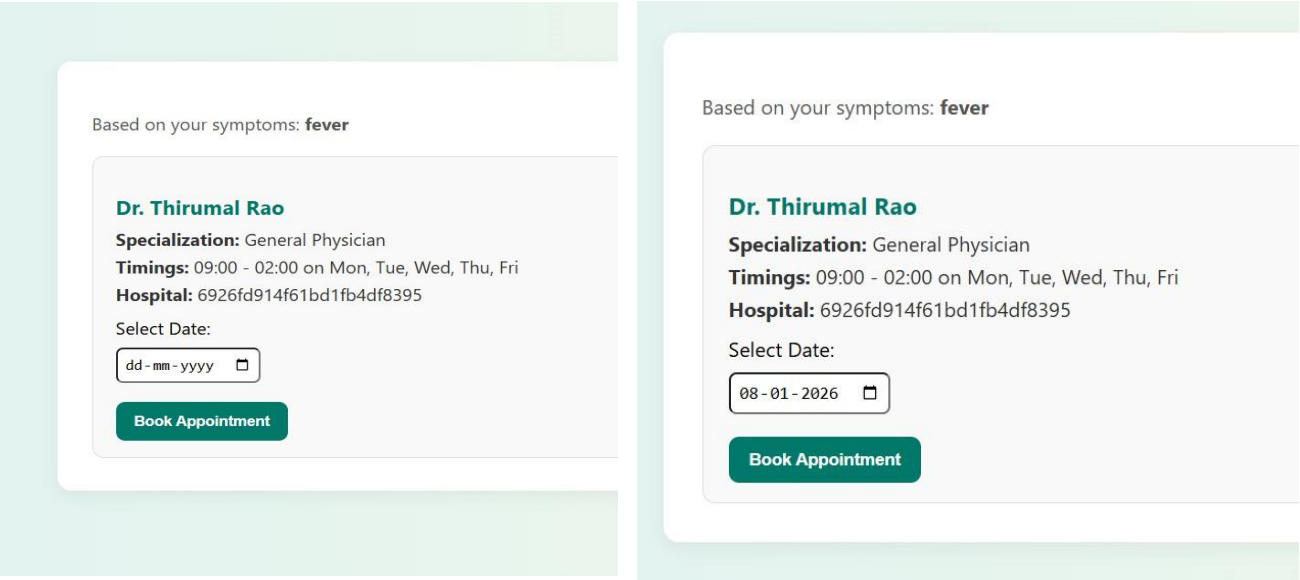


Figure 8.7: Booking appoinment Interface

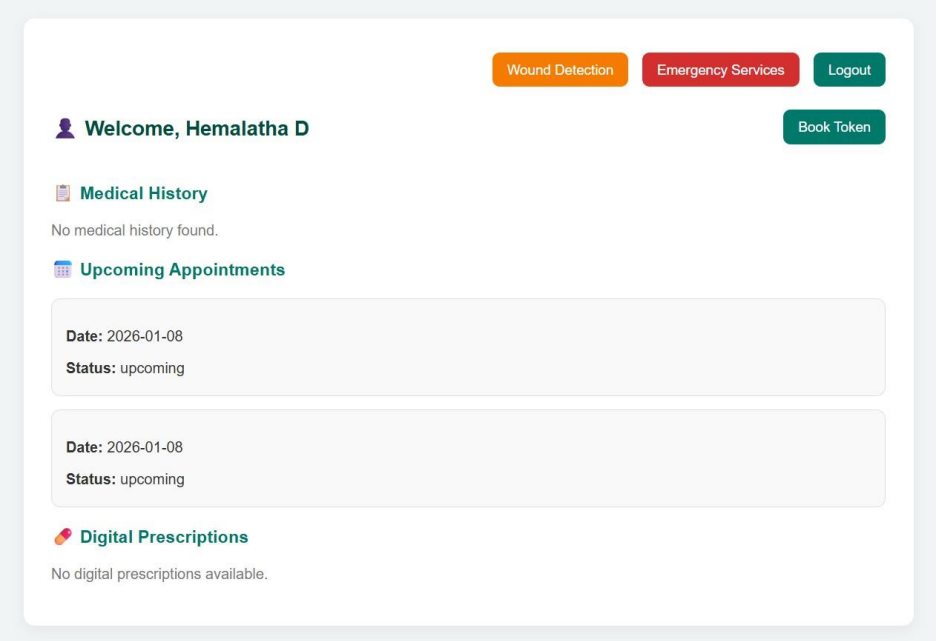


Figure 8.8: User Dashboard

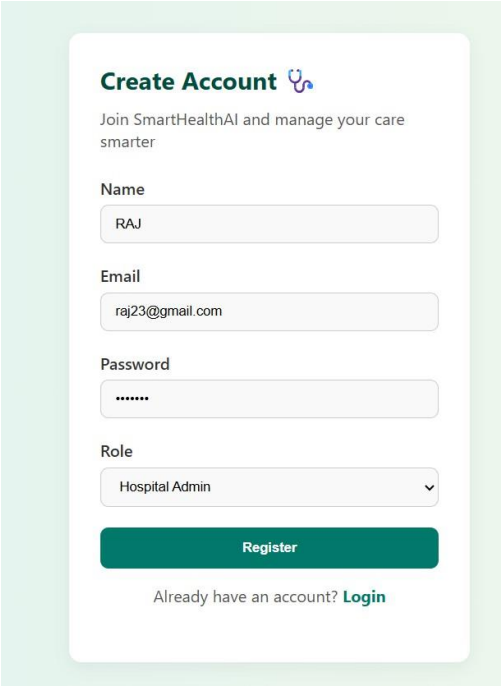


Figure 8.9: Hospial Admin Login

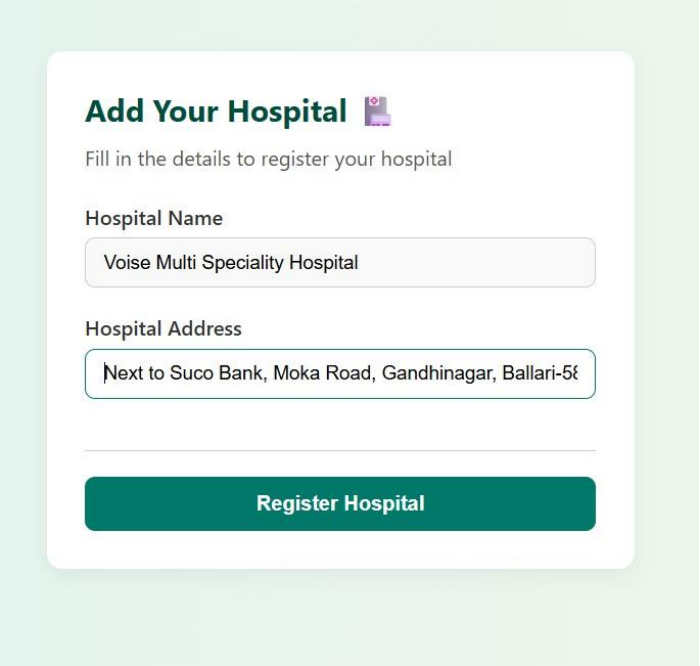


Figure 8.10: Adding Hospital

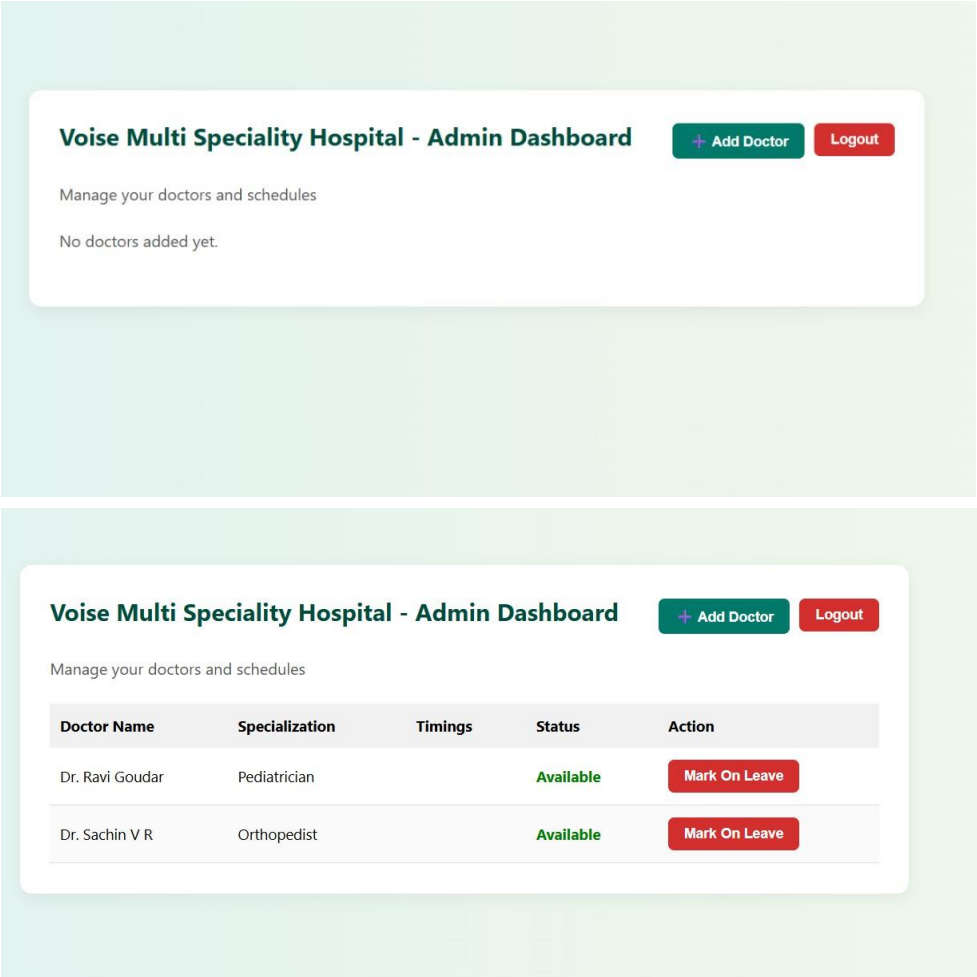


Figure 8.11: Admin Dashboard

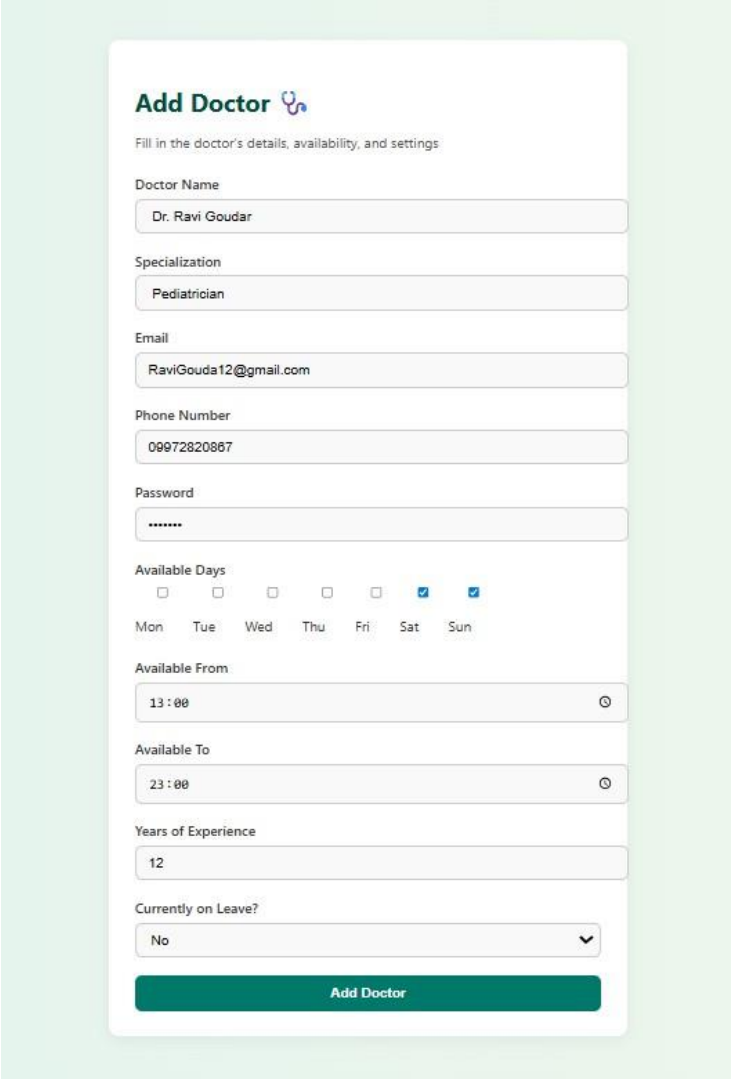


Figure 8.12: Adding Doctor

Figure 8.13 Doctor Dashboard

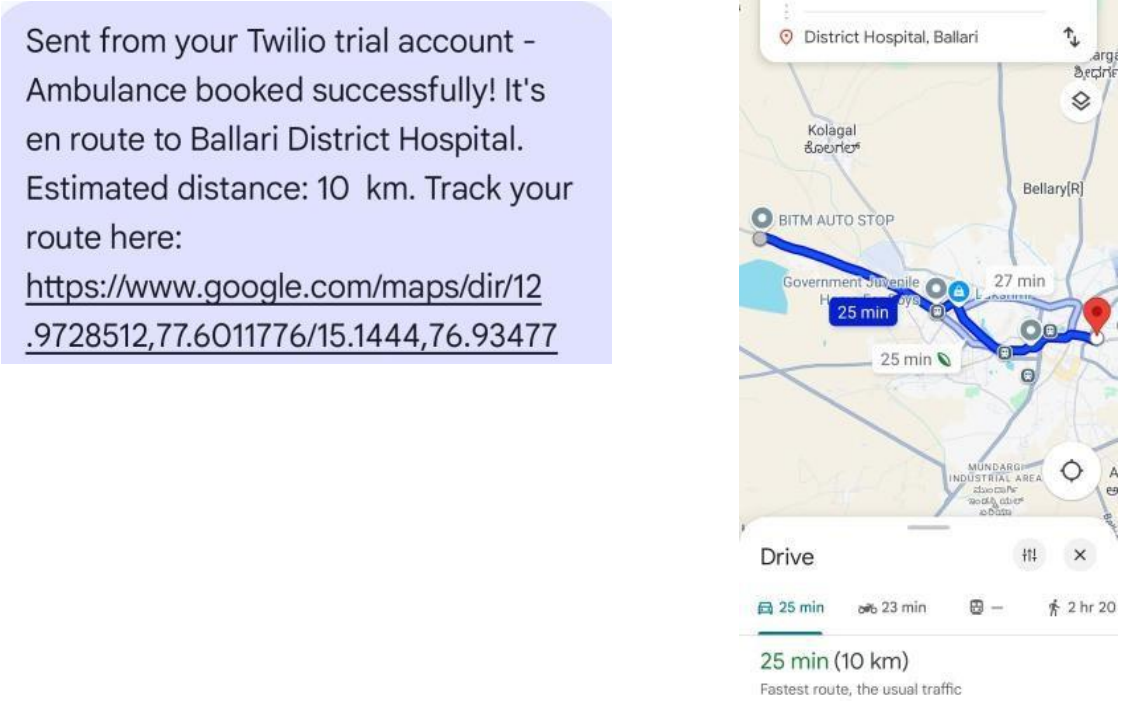


Figure 8.14 Ambulance Dispatch

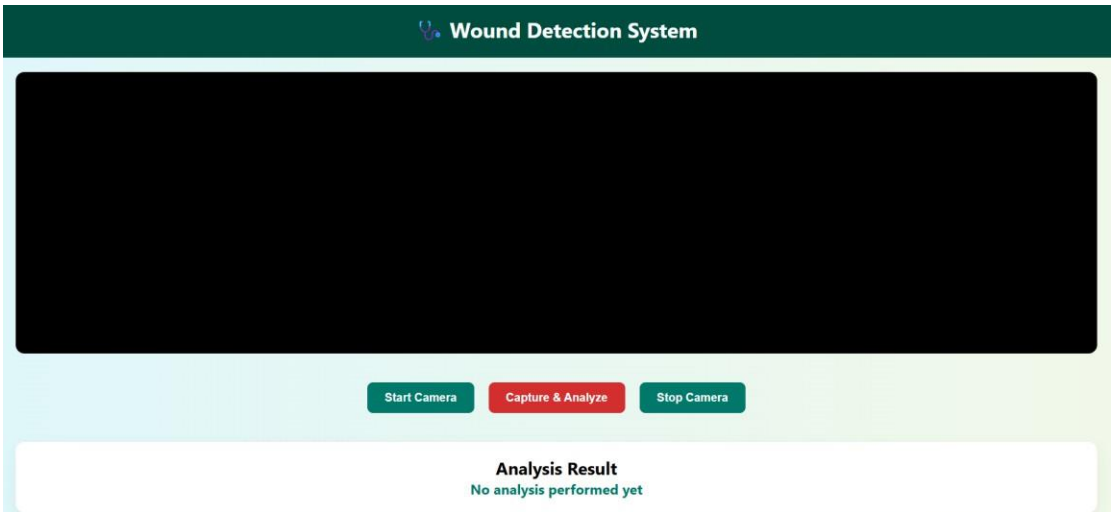


Figure 8.15 Wound Detection Interface

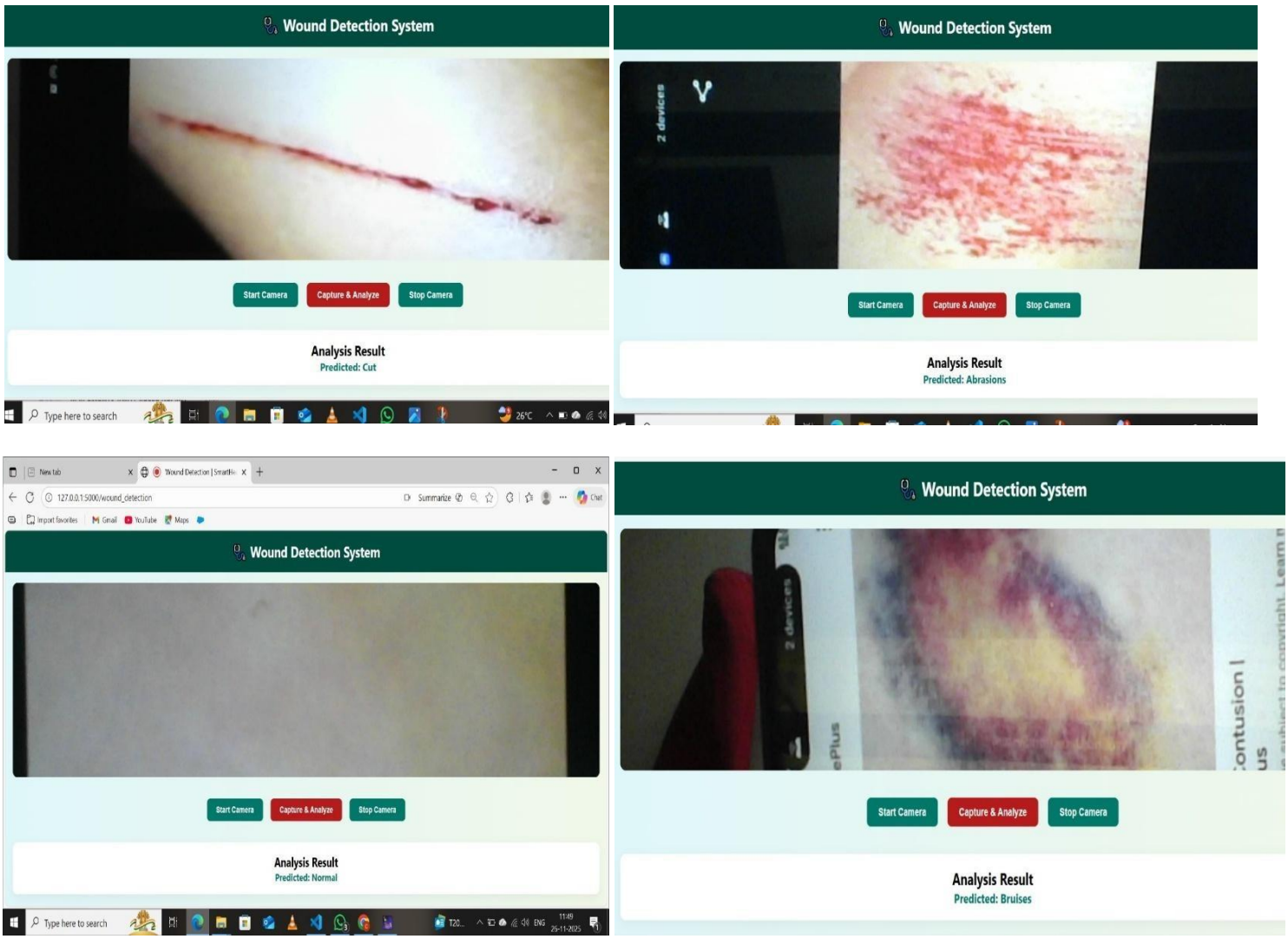


Figure 8.16 Wound Detection outputs

CHAPTER 9

APPLICATIONS AND FUTURE SCOPE

9.1 Applications

Emergency Medical Services (EMS) – Emergency response teams can use the system for real-time accident reporting, ambulance dispatch, and coordination with hospitals and doctors.

Hospitals and Trauma Centers – Hospitals can receive early notifications about incoming patients, view wound severity, allocate doctors in advance, and manage appointments using smart token.

Ambulance Services – Ambulance drivers can use the mobile application for real-time navigation, traffic-aware routing, and live updates about hospital destinations.

Urban Traffic and Smart City Systems – The system can be integrated with smart city infrastructure to improve emergency traffic management and reduce response delays.

Rural and Remote Healthcare Support – In areas with limited medical facilities, RescueNow can help identify nearby hospitals and available doctors quickly, improving access to emergency care.

Public Safety Departments – Police and emergency authorities can monitor accident incidents, ambulance movement, and response effectiveness through centralized dashboards.

9.2 Future Scope

Multilingual Support – Adding support for multiple languages to allow accident reporting, prescriptions, and notifications in regional and international languages.

Advanced AI-Based Diagnosis – Enhancing wound and disease detection using advanced deep learning models for more accurate medical assessment.

Wearable Device Integration – Integrating smartwatches or health sensors to automatically detect accidents and capture patient vitals in real time.

Mobile Application Enhancement – Developing advanced Android and iOS applications for patients, doctors, and ambulance drivers with additional features**.**

Cloud-Based Scalability – Deploying the system on cloud platforms to handle large-scale emergencies, faster processing, and improved system availability**.**

Predictive Emergency Analytics – Using historical accident data to predict accident-prone zones and improve preventive emergency planning.

CONCLUSION

The Rescue Now – Intelligent Emergency Response System presents a comprehensive and technology-driven solution to the critical challenges faced in emergency medical response during road accidents. By integrating real-time accident reporting, GPS-based ambulance dispatch, optimized route planning, AI-based wound detection, and doctor–patient matchmaking, the system significantly reduces emergency response time and enhances treatment preparedness.

The inclusion of features such as prescription recommendations, appointment booking, and smart token generation further improves hospital workflow and patient management, ensuring timely and organized medical care. Real-time monitoring through a centralized admin dashboard enhances coordination among emergency services, hospitals, and medical professionals.

Overall, RescueNow demonstrates how modern technologies like Artificial Intelligence, machine learning, and real-time data integration can transform traditional emergency healthcare systems into smart, reliable, and scalable solutions. By improving efficiency, accuracy, and coordination, the system has the potential to save lives, reduce critical delays, and set a strong foundation for future advancements in intelligent emergency medical services.

REFERENCES

[1] Y. Chen, Y. Li *Etal.*, " A Smart Ambulance System for Efficient Emergency Response", published in a journal or IEEE conference through research on similar topics appears in IEEE access and peer-reviewed venues, in China, in year 2018.

[2] A.Al-Zahrani "A Review of Smart Ambulance Systems," published in Multifactor Authentication for Smart Emergency Medical Response collection, at Saudi Arabia, in year 2021.

[3] Azwati Azmin, Samihah Abdullah, *Et al.,*" Fingerprint Sensor Integration in Smart Healthcare Emergency App: Enhancing Ambulance Navigation through Emergency Route Highlighting," presented at IEEE International Colloquium on Signal processing, At Kuala Lumpur, Malaysia on, March 2024.

[4] Devigayathri P, Amritha Varshini R, *Etal.*, "Mobile Ambulance Management Application for Critical Needs," published in International Conference, at Jalandhar, Punjab, India, on April,2024.

Project Work Phase-II (22CAP76) – CO PO Mapping

Academic Year 2025-26

U.S.N.	Student Name	Guide Name	Project Title
3BR22CA002	A Meghana	Dr. Yeresime Suresh	RESCUE NOW: REVOLUTIONIZING PATIENT-AMBULANCE CONNECTIVITY
3BR22CA019	Hemalatha D		
3BR22CA034	N T Chandrakala		
3BR22CA036	P Naga Sai		

COURSE OUTCOMES(CO'S)

Course Outcomes COx	Description of Course Outcomes
CO1	Demonstrate the ability to apply core computer science concepts to develop practical solutions.
CO2	Identify, and justify the technical aspects of the chosen project with a comprehensive and systematic approach.
CO3	Design, code, and test software solutions for specific problems.
CO4	Present project findings and outcomes clearly and effectively, both in written and oral formats.
CO5	Work as an individual or in a team in development of technical projects.

CO-PO MAPPING

COPO	PO1	PO2	PO3	PO4	PO5	PO6	PO7	PO8	PO9	PO10	PO11	PSO1	PSO2
CO1	3	3	2	3	2	2	1	1	1	2	1	3	2
CO2	3	3	3	3	2	2	1	2	2	3	2	3	2
CO3	3	3	3	2	3	2	1	1	2	2	2	3	3
CO4	2	2	2	2	2	1	1	2	2	3	2	2	2
CO5	1	1	1	1	1	2	1	3	3	3	2	1	2

PROGRAM OUTCOMES AND PROGRAM SPECIFIC OUTCOMES (POs PSOs)

PO1: Engineering Knowledge Apply the knowledge of mathematics, science, engineering fundamentals, and an engineering specialization to the solution of complex engineering problems.
PO2: Problem Analysis Identify, formulate, research literature, and analyze complex engineering problems reaching substantiated conclusions using first principles of mathematics, natural sciences, and engineering sciences.
PO3: Design/Development of Solutions Design solutions for complex engineering problems and design system components or processes that meet specified needs with appropriate consideration for public health and safety, and cultural, societal, and environmental considerations.
PO4: Conduct Investigations of Complex Problems Use research-based knowledge and research methods including design of experiments, analysis, and interpretation of data, and synthesis of the information to provide valid conclusions.
PO5: Modern Tool Usage Create, select, and apply appropriate techniques, resources, and modern engineering and IT tools including prediction and modeling to complex engineering activities with an understanding of the limitations.
PO6: The Engineer and Society Apply reasoning informed by contextual knowledge to assess societal, health, safety, legal, and cultural issues and the consequent responsibilities relevant to professional engineering practice.
PO7: Environment and Sustainability Understand the impact of professional engineering solutions in societal and environmental contexts and demonstrate the knowledge of, and need for, sustainable development.
PO8: Ethics Apply ethical principles and commit to professional ethics and responsibilities and norms of engineering practice.
PO9: Individual and Team Work Function effectively as an individual, and as a member or leader in diverse teams, and in multidisciplinary settings.
PO10: Communication Communicate effectively on complex engineering activities with the engineering community and with society at large, such as being able to comprehend and write effective reports and design documentation, make effective presentations, and give and receive clear instructions.
PO11: Project Management and Finance Demonstrate knowledge and understanding of engineering and management principles and apply these to one's own work, as a member and leader in a team, to manage projects and in multidisciplinary environments.

PSO1: Demonstrate the principles, architecture and organization of computers, embedded systems and computer networks.
PSO2: Develop software applications using advanced technologies to cater the growing needs of industry.

Project Guide

Name: Prof. Vikram Sandeep

Designation: Assistanant Professor

Email: vikramsandeep@bitm.edu.in

Phone No. : 7624941816

Name: Anjum K

USN: 3BR22CA004

Email: anjumq394@gmail.com

Phone No.: 6362790595

Name: Ifrah Anjum M

USN: 3BR22CA020

Email: ifrahanjum3004@gmail.com

Phone No.: 7338470493

Name: Maseera Fathima

USN: 3BR22CA031

Email: maseera4002@gmail.com

Phone No.: 8310062670

Name: Kousar Anjum DL

USN: 3BR23CA402

Email: kousaranjumdl@gmail.com

Phone No.: 7338030208